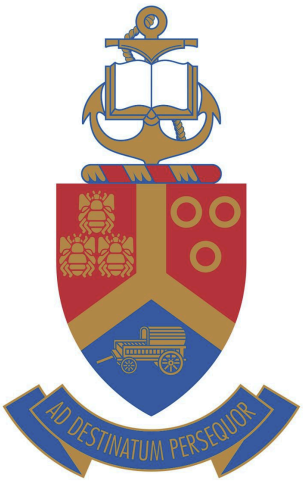
NICOG: MicroRTS
Research Report



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Shahil Parbhoo Narsing - U21746134

Stefan Jansen van Rensburg - U22550055

Heinrich Niebuhr - U22555855

Milan Kruger - U04948123

Supervisors

Issues:
Describe microRTS in more detail, justify why GP is appropriate approach
In lit survey include if GP used in similar game environments
Mix of SA and US english

---

1 Abstract

Real-time strategy (RTS) games such as the simplified research testbed microRTS pose a hard problem for artificial intelligence because their large branching factors and 100-millisecond decision budget demand competent play under tight computational constraints. The approaches that currently dominate microRTS each carry a characteristic weakness: expert-driven rule-based bots are efficient and stable but brittle and poor at generalising, while reinforcement-learning agents can adapt but are expensive to train and prone to catastrophic forgetting. This project investigates an avenue not yet explored in microRTS - evolving a complete, autonomous bot using structure-based genetic programming, whose separate evolution of program structure helps escape the local optima that limit standard genetic programming - and optionally wrapping the evolved bot in a lightweight adapter module that refines its action distribution. The work has two objectives: first, to evaluate whether structure-based genetic programming produces a more competitive and generalisable bot than an ordinary genetic programming baseline and than existing reinforcement-learning and rule-based bots; and second, to determine whether the adapter module further improves the bot's performance and generalisation across varying RTS scenarios. Effectiveness is established by proof of demonstration, comparing win rates against held-out opponents and maps.

1.1 Keywords

microRTS; real-time strategy games; structure-based genetic programming; genetic programming; adapter module; reinforcement learning; game tree search; autonomous agents; game artificial intelligence; generalisation

2 Introduction

2.1 Area and Topic

Real-time strategy (RTS) games are a genre of video game in which players manage resources, construct buildings, and train and command units continuously in real time to achieve strategic objectives, most commonly the destruction of all the opponent's units. RTS games are notorious for their large branching factors, which pose a substantial challenge to game-tree search algorithms [1]. MicroRTS is a simplified RTS game with a single resource type, a small set of unit types, two building types and simplified terrain; this simplicity is what makes it well suited to benchmarking new artificial intelligence (AI) algorithms. It was created by Santiago Ontañón in 2013 for AI research purposes [1]. An annual microRTS competition is held, the most recent edition of which has two tracks: a classic track, in which each bot has only 100 milliseconds to decide its actions per game cycle, and an informal track, in which the decision time may be adjusted [2]. This proposal concerns only the classic track and the default microRTS configuration, which is deterministic and fully observable.

2.2 Brief Tour of Approaches

A range of approaches has been used to build competitive microRTS bots, each with its own strengths and weaknesses. Search-based methods such as Monte Carlo Tree Search address the branching factor by sampling the action space rather than enumerating it. Reinforcement learning (RL) instead learns a policy in advance from experience [3], [4], and a recent variation augments it with an adapter module, inspired by natural language processing, that refines a fixed base policy and can be applied to any algorithm that outputs a probability distribution over actions [4]. Rule-based AI, in which a human encodes strategy explicitly, currently produces the strongest competitors. Large-language-model-based AI has also been explored [5], but it belongs to the informal track and is therefore outside the scope of this proposal. One avenue that remains entirely unexplored is the use of genetic programming to evolve a bot; this project pursues that avenue, and goes further by using structure-based genetic programming, which helps escape local optima.

2.3 Problem Statement

The approaches above present a recurring tension, examined in detail in Section 3: the best-performing bots are expert-driven rule-based systems which that are brittle and generalise poorly because their behaviour is fixed at design time, while the learning-based approaches are expensive to train and prone to catastrophic forgetting, in which further training erodes competence the bot previously had [4]. This project investigates whether structure-based genetic programming, optionally combined with an adapter module, can produce a competitive and generalisable microRTS bot at lower computational cost than reinforcement learning and without the unoptimised hand-engineered rule-based systems.

2.4 Aims and Objectives

The project has two objectives. The first is to evaluate the effectiveness of structure-based genetic programming against previous reinforcement-learning and expert-driven rule-based bots. The second is to compare structure-based genetic programming with an adapter-module-based version of itself [4], to determine whether the adaptation mechanism enhances generalisation across varying RTS scenarios. To isolate the contribution of the structure-based component, an ordinary genetic programming bot is also evaluated as a baseline against the structure-based variant. Meeting these objectives requires four tasks: to design and implement a genetic programming algorithm that produces an AI bot; to extend it to a structure-based genetic programming algorithm; to implement the adapter module; and to measure the win rate of the ordinary and structure-based genetic programming bots, each with and without the adapter.

2.5 Section Breakdown

The remainder of this proposal is organised as follows. Section 3 surveys the literature on the approaches introduced above and analyses the gap this project addresses; Section 4 describes the research methodology; Section 5 sets out the workplan; and Section 6 lists the references.

3 Literature Survey

3.1 MicroRTS as a Benchmark

MicroRTS is a minimalist, two-player, zero-sum real-time strategy game created by Santiago Ontañón in 2013 specifically as a testbed for artificial intelligence research [1]. It is played on a rectangular grid and, in its default configuration, is deterministic and fully observable, with all units able to issue commands simultaneously in continuous time; unit actions are durative, meaning each action occupies several game frames before it completes. The game retains the defining mechanics of commercial RTS games while stripping away their incidental complexity: there is a single resource type, gathered by worker units and returned to a base, which is spent to produce further workers or to construct barracks; barracks in turn train three combat unit types - light, heavy and ranged - that differ in durability, speed and attack range. A game is won by destroying all of the opponent's units and buildings. This deliberate simplification preserves the core AI challenges of RTS play - resource management, unit production, long-term strategic planning and real-time combat - while removing the high computational and engineering costs of working with a commercial title, which is what makes microRTS suitable for benchmarking new algorithms [1]. To provide a standardised basis for comparison, an annual microRTS competition is held in which bots compete under strict time limits; in the classic track each bot has 100 milliseconds per game cycle to choose its actions, making computational efficiency a first-class design concern [6]. This project targets that classic track and the default deterministic, fully observable configuration.

3.2 Search-Based Approaches

The earliest competitive microRTS bots were based on game-tree search, most notably Monte Carlo Tree Search (MCTS). MCTS incrementally builds a partial game tree in which each node represents a game state and each edge an action; every node records how many times it has been visited and the average reward obtained from it. From the root, a tree policy selects child nodes until an unexpanded node is reached and added to the tree, after which a playout selects actions - randomly or by some heuristic - until a terminal state or a time limit is reached. The resulting reward is then propagated back up the visited nodes, updating their statistics, and when the computation budget is exhausted the most promising action from the root is played [7]. A 2017 competition winner used MCTS with Naïve Sampling, a sampling scheme designed for microRTS's enormous action space [1].

The difficulty MCTS must overcome is that a player issues actions to many units at once, so the number of combinations of unit-actions explodes. This is naturally framed as a combinatorial multi-armed bandit (CMAB) problem, in which the agent must allocate a limited sampling budget across an action space that is the product of each unit's choices [1]. Subsequent work refined this framing - incorporating linear side information to share learning across related actions [8] and examining the robustness of CMAB algorithms under varying conditions [9]. Two findings from this line of work are notable: explicitly modelling the interdependence between unit-actions,

rather than treating them independently, improves performance, and exposing the sampler to diverse training contexts is important for the results to hold across situations [8], [9]. Across these methods, evaluation of non-terminal states relies on a heuristic. A common choice is microRTS's built-in SimpleSqrtEvaluationFunction3, inspired by the LTD2 function of Churchill et al. [10], which scores each player by summing the resource cost of her units weighted by the square root of their health, then normalises the two scores into the interval [-1, 1] as $E(s) = 2 \cdot score(max) / (score(min) + score(max)) - 1$ [1]. Such heuristics are hand-designed, but the evaluation function can also be learned: convolutional neural networks have been trained to evaluate RTS game states directly from the grid [11], offering a data-driven alternative that this project later draws on as a candidate fitness signal. The shared limitation of all search-based methods is that their quality is bounded by the sampling they can perform within the per-cycle budget, so they search only shallowly and degrade as the state and action spaces grow.

3.3 Reinforcement Learning Approaches

Reinforcement learning (RL) approaches replace per-cycle search with a policy learned in advance from experience, shifting the computational cost from play time to training time. The Gym-μRTS environment made this practical by wrapping microRTS in a standard RL interface and introducing techniques suited to the domain - among them invalid-action masking, a shaped reward function, and the GridNet representation, which computes actions for all units in a single network evaluation by producing action logits at every grid cell [3]. These advances allowed Proximal Policy Optimization (PPO), a widely used policy-gradient algorithm, to be trained on full microRTS games. Refinements such as a partial advantage estimator have since been proposed to improve PPO's stability and sample efficiency in this setting [12].

The strongest demonstration of RL in microRTS is RAISocketAI, the first deep RL agent to win the competition, taking the 2023 title [13]. Its success, however, exposes the approach's costs plainly. Training required on the order of seventy GPU-days, and transfer learning to specific maps was essential to its winning performance - the agent in fact selects among seven policy networks depending on the map and available compute. Subsequent work showed that behavioural cloning from existing bots followed by PPO fine-tuning can bootstrap a competitive agent more economically and from sparse win-loss rewards alone, reducing the need for handcrafted reward shaping [13]. Even so, two weaknesses persist and directly motivate this project: the training cost remains high, and RL agents are prone to catastrophic forgetting, in which continued training on new situations erodes competence on ones previously mastered [4].

3.4 Expert-Driven Rule-Based (State of the Art)

Despite the progress of search and learning methods, the strongest microRTS bots remain expert-driven rule-based (EDRB) systems, in which a human encodes strategy directly as explicit rules and scripts. The 2024 competition winner, Tactical Manager AI (TMA), is rule-based and uses A* pathfinding; it maintains four strategies - two offensive and two defensive - and switches between them according to the game state, with the thresholds governing those switches adapting in real time [14]. TMA was not entered in 2025, where the

winners were two EDRB bots used as benchmarks, Coacai and Mayari. The enduring strength of these systems is that they are efficient, interpretable and stable: they run comfortably within the 100 ms budget and behave predictably. Their weakness is equally characteristic - being fixed at design time, they are brittle and generalise poorly, performing only as well as their authors anticipated the situations they would face. This combination of high performance and poor adaptability with TMA as the exception is the benchmark any new approach must improve upon, and the limitation it must address.

3.5 Evolutionary Computation in RTS

Evolutionary computation refers to a family of optimisation techniques inspired by natural selection, in which a population of candidate solutions is repeatedly evaluated against a fitness function, and the better-performing candidates are recombined and mutated to produce the next generation. Over many generations the population converges toward high-quality solutions without any explicit model of the problem being required. Two variants are relevant here. A genetic algorithm (GA) evolves fixed-length strings of parameters, whereas genetic programming (GP) evolves variable-sized programs, typically represented as trees of functions and terminals, so that the structure of the solution itself is discovered rather than just its parameters [15].

Within RTS games, evolutionary methods have so far been applied to isolated sub-components of an agent rather than to the agent as a whole. Liu, Louis and Ballinger used a genetic algorithm to evolve effective microbehaviors - the short-term positioning, movement and targeting decisions that govern combat - in the RTS game StarCraft [16]. Their agent represented terrain and enemy positions using influence maps and potential fields, and encoded each unit's combat behaviour in fourteen parameters that the GA tuned. The resulting ECSLBot outperformed two state-of-the-art bots in skirmish scenarios, and, importantly, the parameters evolved in one scenario continued to perform well in unseen scenarios. This suggests that evolved behaviours can generalise, which is precisely the property that rule-based and reinforcement-learning bots struggle to achieve.

A second line of work evolves not behaviour directly but the evaluation function that guides another algorithm. Neufeld, Mostaghim and Perez-Liebana studied a hybrid planner in microRTS that combined a Hierarchical Task Network for high-level strategy with Monte Carlo Tree Search for micromanagement, where the interface between the two levels was a set of evaluation functions expressed as weighted sums of game features [17]. They used an evolutionary algorithm to tune these weights automatically, and the agent using the evolved evaluation functions improved on one using hand-designed weights when measured against state-of-the-art controllers. This demonstrates that evolutionary search can replace manual engineering of the heuristics that other approaches depend on.

Taken together, these works establish that evolutionary computation is effective for parts of an RTS agent: combat micro-management and the evaluation functions that steer search. What has not been attempted is the use of evolution to produce a complete, autonomous microRTS

bot - one that handles resource gathering, production, and combat decisions within a single evolved program. Genetic programming is the natural candidate for this, because its tree-based representation can express conditional, hierarchical decision logic of the kind a full bot requires, rather than only a fixed vector of parameters. A known weakness of standard GP, however, is its tendency to converge prematurely on local optima. Structure-based genetic programming addresses this by evolving the program's structure separately from its contents, which has been shown to improve search and escape local optima in other domains (Pillay, [year]). Its application to microRTS, and to RTS bot construction more generally, remains unexplored, and it is this gap that the present project addresses.

3.6 The Adapter-RL Approach

A recent approach aims to combine the interpretability and low cost of rule-based systems with the adaptive capacity of reinforcement learning. Adapter-RL, introduced by Jin, Slabaugh and Lucas, keeps an existing base agent fixed and attaches a small, separately trained neural module - the adapter - that adjusts the base agent's decisions for a specific task [4]. The concept is borrowed from natural language processing, where small adapter modules specialise a large fixed model to new tasks. The base policy may be of essentially any kind - neural, scripted, heuristic, symbolic or pathfinding-based - provided it can be expressed as a probability distribution over actions; the base agent's output is converted to such a distribution using a softmax with a temperature parameter, and the adapter then produces an adjustment to it. The authors report that an additive adjustment outperforms a multiplicative one, and that performance is robust across a broad range of temperature coefficients, roughly $\tau \in [1/1000, 1/100]$, which speaks to the stability of the method.

The reported benefits are directly relevant to this project. Because the adapter is small and the base agent already supplies competent behaviour, the adapter is learned far faster than a policy trained from scratch with PPO, requires far fewer updates, and often outperforms the original base agent, while explicitly mitigating catastrophic forgetting [4]. Two caveats are noted: the quality of the adapted agent depends heavily on the quality of the base policy, and the approach has been demonstrated chiefly on neural and scripted base agents. Because the adapter is agnostic to the base agent's internals and requires only an action distribution, it can in principle wrap a genetic-programming bot - a possibility this project investigates as its second objective. It should be noted that, since a genetic-programming bot is a fixed policy once evolved, the adapter benefits most relevant here are its rapid adaptation and its refinement of an already competent base, rather than the mitigation of catastrophic forgetting, which is a hazard of continually gradient-trained agents rather than of an evolved program.

3.7 Critical Analysis

The approaches reviewed above each address the central difficulties of microRTS - a vast combinatorial action space, durative simultaneous actions, and a 100 ms decision budget - but each does so with a weakness, and it is the pattern across these weaknesses that motivates this project.

3.7.1 Gap (Type 1)

Search-based methods such as Monte Carlo Tree Search with Naïve Sampling and combinatorial multi-armed bandit formulations cope with the branching factor by sampling rather than enumerating the action space, but their quality is fundamentally bounded by the computation available per game cycle [1]. Under the classic-track constraint they can search only shallowly, and their performance degrades as map size and unit counts grow. Deep reinforcement learning removes the per-cycle search but relocates the cost to training: RAISocketAI, the first DRL agent to win the competition, required roughly seventy GPU-days and depended on transfer learning to individual maps to remain competitive [13]. RL agents also suffer catastrophic forgetting, in which learning a new task erodes competence on one already mastered [4], undermining the generalisation that a single deployable bot requires. Expert-driven rule-based bots are the current state of the art - TMA won in 2024, and Coacai and Mayari in 2025 - and they are efficient, interpretable and stable, but this performance is bought with hand-engineering and is brittle: their behaviour is fixed at design time and generalises poorly to situations their authors did not anticipate.

Evolutionary computation offers a route around these trade-offs, since it requires neither expensive gradient-based training nor exhaustive hand-engineering, and evolved solutions have shown the capacity to generalise to unseen scenarios [16]. Yet in RTS games it has so far been applied only to isolated sub-components - combat microbehaviors [16] and the evaluation-function weights that steer a planner [17] - and never to evolve a complete autonomous bot. Genetic programming, whose tree-based representation can express the hierarchical, conditional decision logic a full bot requires, has not been applied to microRTS at all. This is the gap the project addresses: whether genetic programming, and specifically structure-based genetic programming, which mitigates the premature convergence of standard GP [18], can produce a competitive microRTS bot.

3.7.2 Borrowed Methods and Divergence (Type 2)

The project builds on three elements drawn from the reviewed work. From the evolutionary literature it adopts the principle of evolving an agent against a fitness signal rather than hand-coding or gradient-training it [16], [17]. As that fitness signal it draws on the evaluation functions established in prior microRTS work, such as the resource-and-health-weighted SimpleSqrtEvaluationFunction3 [1] and learned CNN-based state evaluators [11]. From the reinforcement-learning literature it adopts the Adapter-RL module, which refines a frozen base policy that outputs an action distribution, trains far faster than learning a policy from scratch, and explicitly counteracts catastrophic forgetting [4].

Where the project diverges is in scope and combination. Unlike Liu et al. and Neufeld, who evolve parameters or weights for a sub-component, this work uses structure-based genetic programming to evolve a complete bot, and unlike standard GP it evolves program structure separately to escape local optima. And whereas Adapter-RL has been demonstrated wrapping neural and scripted base policies, this project tests it on an evolved GP policy - a base agent

type it has not previously been applied to - to determine whether the adapter improves the generalisation of a structure-based GP bot across varying microRTS scenarios. These choices yield the project's two objectives: first, to determine whether structure-based genetic programming produces a more competitive and generalisable bot than an ordinary GP baseline and than existing reinforcement-learning and expert-driven rule-based bots; and second, to determine whether wrapping that bot in an adapter module further improves its performance. The two objectives are evaluated across four configurations - an ordinary GP bot, a structure-based GP bot, and each of these wrapped with the adapter.

4 Research Methodology

This section describes how the research will be conducted and validated, not the design of the bot itself. The genetic programming algorithm and adapter are the artefacts; what follows is the method by which their effectiveness is established.

4.1 Research Method

The research follows a proof-by-demonstration approach. Because the central claim - that structure-based genetic programming can produce a competitive microRTS bot, and that an adapter module improves it - cannot be settled analytically, it is established empirically by constructing the artefacts and measuring their performance against existing bots under controlled conditions. The methodology therefore centres on building four agent configurations and comparing them: an ordinary genetic programming bot, a structure-based genetic programming bot, and each of these wrapped with the adapter module. The ordinary-versus-structure-based comparison isolates the contribution of the structural component, and the with-versus-without-adapter comparison isolates the contribution of the adapter, so that each objective is answered by a clean controlled contrast rather than a single end-to-end result.

4.2 Experimental Design

Performance is measured primarily by win rate, the proportion of games an agent wins over a large number of matches, since victory in microRTS is unambiguous (the destruction of all opposing units and buildings). As a secondary metric, the computational cost of producing each bot - the wall-clock evolution or training time and the hardware used - is recorded, since a central claim of the project is that genetic programming reaches competitive play far more cheaply than reinforcement learning, and this claim can only be supported if that cost is measured. Each of the four configurations is evaluated against a panel of opponents drawn from the approaches surveyed in Section 3: expert-driven rule-based bots, using the 2025 competition benchmarks Coacai and Mayari and, where available, the 2024 winner TMA; a reinforcement-learning baseline; and the configurations against one another. The reinforcement-learning baseline is a published, pre-trained Gym-μRTS PPO agent rather than one trained in-house: reproducing a competitive deep RL agent would require on the order of the seventy GPU-days reported for RAISocketAI [13], which is infeasible on a single consumer GPU. Relying on a released agent is appropriate here, and it reinforces the project's motivation, since the inability of an ordinary research setup to afford such training is exactly the cost problem the genetic programming approach is intended to avoid.

To ensure the experiment measures generalisation rather than memorisation, the opponents and maps are partitioned into disjoint training and evaluation sets. The genetic programming bots are evolved - and the adapter is trained - against a set of training opponents drawn from microRTS's built-in scripted bots (such as WorkerRush, LightRush and a Naïve MCTS agent) on the training maps only, while all reported win rates are obtained against the disjoint held-out

evaluation panel described above (Coacai, Mayari, TMA where available, and the pre-trained reinforcement-learning agent) on held-out maps of varying size that were never seen during evolution or adapter training. Without this separation, a bot that had merely over-fitted to its training opponents would be indistinguishable from one that had genuinely generalised. All games use the default deterministic, fully observable configuration under the classic-track 100 ms decision limit, so that results are comparable with published competition outcomes. The experiment is structured as an ablation: holding the evaluation opponents, maps and match counts fixed while varying only whether the structural component or the adapter is present allows any change in win rate to be attributed to that single factor. Results are reported both per held-out map and aggregated.

4.3 Stochastic Considerations

Both the method and the testbed contain randomness. Genetic programming is stochastic - its initial population, crossover and mutation depend on a random seed - so a single evolutionary run is not evidence; the evolution will be repeated across many independent seeds, and results reported as distributions (means with variance, or box plots) rather than single figures. Although the default game is deterministic, the agents' own sampling and the choice of starting positions introduce variation between matches, so each pairing is played over many games with sides swapped to remove any first-player advantage. The feasible number of seeds is bounded by CPU time rather than GPU memory, because each fitness evaluation is a microRTS game simulation that runs on the processor; the evolutionary runs are therefore parallelised across CPU cores, and the seed count is set to the largest value that fits the available compute budget. Reporting distributions across seeds and matches, rather than best-case runs, guards against drawing conclusions from a single lucky evolution.

4.4 Statistical Tests

Differences in win rate are assessed for statistical significance rather than reported as raw numbers. Because the primary outcome is a proportion of wins, comparisons between two configurations use an appropriate test for proportions, and comparisons of performance distributions across seeds use a non-parametric test such as the Mann-Whitney U test, which makes no assumption that the results are normally distributed. A significance threshold is fixed in advance ($\alpha = 0.05$), and where many pairwise comparisons are made a correction for multiple comparisons is applied so that the overall error rate is controlled. This ensures that a claimed improvement from the structural component or the adapter reflects a real effect and not sampling noise.

4.5 Refinement Strategy

The first implementation is unlikely to be the best, so the methodology includes a deliberate refinement loop. If a configuration underperforms - or performs well and is being pushed further - the elements available for adjustment are the fitness function, the program representation (the function and terminal sets the GP can use), the GP control parameters (population size,

generation count, crossover and mutation rates), and, for the adapter, the temperature coefficient and the choice of additive over multiplicative adjustment. Candidate fitness signals range from the built-in SimpleSqrtEvaluationFunction3 [1] to a learned CNN-based state evaluator [11]. The two halves of the refinement loop differ in cost: adapter refinements are cheap, since the adapter trains with far fewer updates than a policy learned from scratch and its temperature and adjustment settings can be swept quickly on the GPU, whereas GP refinements are bounded by CPU game-simulation time and so are explored more selectively. Each refinement is treated as a controlled change measured against the same held-out evaluation panel, so that the effect of the change is itself evidence rather than guesswork, and the refinement history is recorded as part of the results.

4.6 Technical Specifications

The work is built on the original microRTS framework, which is implemented in Java and exposes the game engine, the competition maps and the benchmark bots (Coacai, Mayari and others) used as opponents; the Gym-μRTS interface [3] is used where the pre-trained reinforcement-learning baseline or the adapter's neural component requires a Python and PyTorch environment. The genetic programming component is implemented using an established GP library appropriate to the chosen language. Experiments are run on a single workstation equipped with an NVIDIA GTX 5070 GPU (12 GB VRAM) with CUDA, used together with PyTorch for the adapter and for any neural fitness evaluator. The workloads divide cleanly across the hardware: the genetic programming evolution is CPU-bound and embarrassingly parallel, since each fitness evaluation is an independent Java game simulation, while the GPU is reserved for adapter training and CNN-based state evaluation. The full hardware specification (CPU, GPU and memory) and all software versions are recorded, and to support reproducibility the random seeds, configuration files, evolved programs and match logs are version-controlled and archived, so that any reported result can be regenerated.

5 Workplan

The project is planned as a sequence of phases that follow the dependencies between the artefacts: the environment and baselines must work before an ordinary GP bot can be evolved, the ordinary GP bot must exist before the structure-based extension, and a working bot must exist before the adapter can wrap it. Two go/no-go gates govern progression — G1 confirms the evolved bot meets the real-time decision budget before the approach is scaled up, and G2 confirms the GP policy can expose an action distribution before the adapter phase is committed to. Evaluation runs continuously rather than as a single terminal block: each artefact is benchmarked as it is produced. Writing begins once the early phases complete, and contingency is concentrated at the two highest-risk points — after Phase 3 and before submission — rather than spread evenly across the project.

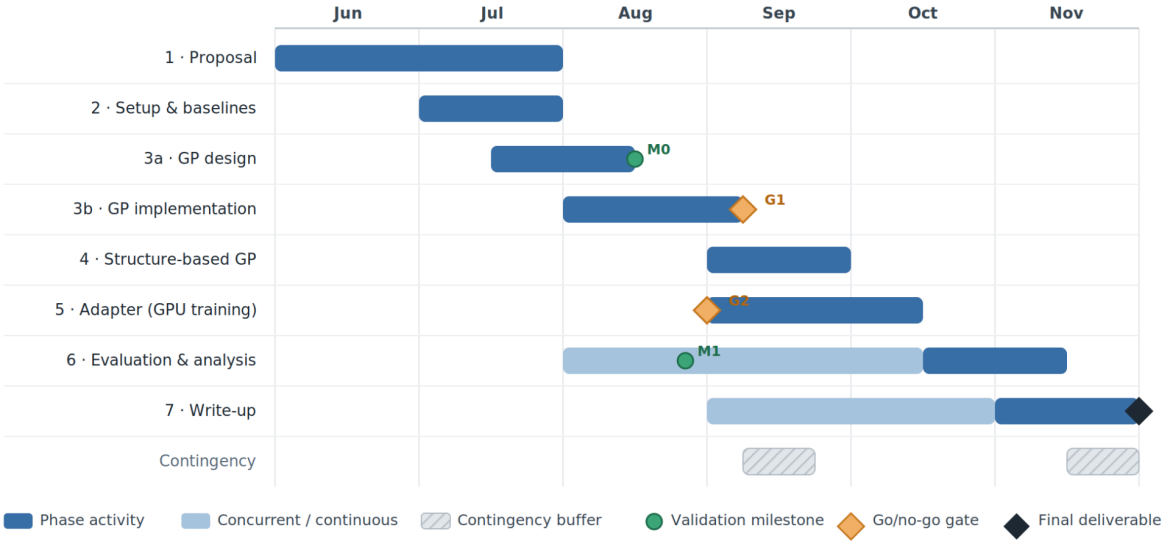
Phase	Activity	Milestone / Deliverable	Target
1 · Proposal	Finalise the literature survey, methodology and references; address review feedback.	Approved research proposal.	Jun-Jul
2 · Environment, baselines & reproducibility	Set up microRTS and Gym-μRTS; integrate the benchmark bots (Coacai, Mayari) and the pre-trained RL agent; reproduce known baseline win rates; fix the training/evaluation opponent and map splits; establish version control, seed logging and configuration management.	Working experimental harness with reproduced baselines and reproducibility tooling in place.	Jul
3a · GP design	Design the function and terminal sets, the fitness function (including a dense shaped signal so early evolution has a non-flat landscape) and the evolutionary loop.	Documented primitive set and fitness design.	Jul-Aug
M0 — validation milestone	*Hand-author one program from the chosen primitives that plays a complete game, confirming the function/terminal set can express a viable bot before evolution is relied upon.*	*Hand-authored reference bot playing a complete game.*	*early Aug*

3b · GP implementation & first bot	Implement the evolutionary loop and evolve an ordinary GP bot that plays complete games; profile per-cycle inference time; evaluate against a graded opponent ladder (random → light rush → competitive scripted bots).	Ordinary GP bot competing on the training maps.	Aug - early Sep
M1 — validation milestone	*Evolved bot beats a weak baseline (e.g. WorkerRush) — the first rung of the opponent ladder.*	*First ladder rung cleared; early success signal.*	*late Aug*
G1 — go/no-go gate	*Confirm the evolved bot plays complete games within the 100 ms/cycle decision budget; apply parsimony pressure / tree-depth caps if it is exceeded.*	*Timing report confirming real-time compliance.*	*early Sep*
4 · Structure-based GP	Extend the GP to structure-based GP, evolving program structure separately from contents; re-evaluate on the opponent ladder against the ordinary GP bot.	Structure-based GP bot with comparative results.	Sep
G2 — go/no-go gate	*Confirm the GP policy can emit an action distribution suitable for the adapter (a softmax over per-unit action scores). Decision point for Phase 5.*	*Validated GP-to-distribution interface.*	*early Sep*
5 · Adapter module (GPU training)	Implement the adapter wrapping the GP policy; train it against the training opponents on the GPU; tune the temperature and the adjustment type.	Adapter-augmented GP bot.	Sep - mid Oct
6 · Evaluation & analysis	Benchmark each artefact as it is produced (ordinary GP in August, structure-based in September, adapter in October); then run the full ablation over many seeds on held-out opponents and maps and apply the statistical tests; iterate via the refinement loop, capped at two iterations / a two-week window.	Continuous benchmark results, then the complete ablation with significance testing.	Aug - Nov (full ablation mid-Oct - mid-Nov)

7 · Write-up	Draft the methodology and implementation chapters as those phases complete; write the results, discussion and conclusions; prepare the presentation and reproducibility archive.	Final dissertation and presentation.	Sep - Nov
Contingency	Located buffer for implementation overruns and refinement: one block after Phase 3 (the phase most likely to overrun) and one before submission.	—	early-mid Sep; mid-late Nov

MicroRTS Project — Revised Workplan Timeline

June–November · Phase 3 split, continuous evaluation, write-up overlapping, buffer placed at the two highest-risk points



M0 Hand-author one program from the chosen primitives that plays a complete game — validates the function/terminal set before trusting evolution.
M1 Evolved bot beats a weak baseline (e.g. WorkerRush) — first rung of the opponent ladder and an early success signal.
G1 Evolved bot plays complete games within the 100 ms/cycle budget; profile and apply bloat control if exceeded.
G2 GP policy can emit an action distribution — go/no-go gate for the adapter (Objective 2) before committing to Phase 5.

Adapter training (5) and the full ablation (6) are sequenced through October so they do not contend for the single GPU; evaluation runs continuously from August.

Changes from the original plan: Phase 3 split into design / implementation (design overlaps Phase 2 in July); evaluation made continuous; write-up started in September; reproducibility folded into Phase 2; contingency placed after Phase 3 and before submission rather than "throughout".

6 Bibliography

REFERENCED
(cited in the proposal text).

Browne, C.B., Powley, E., Whitehouse, D., Lucas, S.M., Cowling, P.I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S. and Colton, S. (2012) 'A survey of Monte Carlo tree search methods', IEEE Transactions on Computational Intelligence and AI in Games, 4(1), pp. 1-43.

Churchill, D., Saffidine, A. and Buro, M. (2012) 'Fast heuristic search for RTS game combat scenarios', Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 8(1), pp. 112-117.

Goodfriend, S. (2024) 'A competition winning deep reinforcement learning agent in microRTS', 2024 IEEE Conference on Games (CoG). IEEE, pp. 1-8.

Huang, S., Ontañón, S., Bamford, C. and Grela, L. (2021) 'Gym-μRTS: toward affordable full game real-time strategy games research with deep reinforcement learning', 2021 IEEE Conference on Games (CoG). Copenhagen, Denmark: IEEE, pp. 671-678.

Jin, Y., Slabaugh, G. and Lucas, S. (2023) 'ADAPTER-RL: adaptation of any agent using reinforcement learning', arXiv preprint arXiv:2311.11537.

Jin, Y., Song, X., Lucas, S. and Slabaugh, G. (2024) 'Partial advantage estimator for proximal policy optimization', IEEE Transactions on Games. doi:10.1109/TG.2024.3408298.

Komenda, A., Shleyfman, A. and Domshlak, C. (2014) 'On robustness of CMAB algorithms: experimental approach', in Cazenave, T., Winands, M.H.M. and Björnsson, Y. (eds.) Computer Games. Cham: Springer International Publishing, pp. 16-28.

Koza, J.R. (1992) Genetic Programming: On the Programming of Computers by Means of Natural Selection. Cambridge, MA: MIT Press.

Liu, S., Louis, S.J. and Ballinger, C.A. (2016) 'Evolving effective microbehaviors in real-time strategy games', IEEE Transactions on Computational Intelligence and AI in Games, 8(4), pp. 351-362.

Mazza, A. (2024) TMA: Tactical Manager AI [Software]. Available at: https://github.com/MazzaAlessandro/TMA (Accessed: 13 June 2026).

microRTS AI Competition (no date) microRTS AI competition - rules. Available at: https://sites.google.com/site/micrortsaicompetition/rules (Accessed: 30 May 2026).

Neufeld, X., Mostaghim, S. and Perez-Liebana, D. (2019) 'Evolving game state evaluation functions for a hybrid planning approach', 2019 IEEE Conference on Games (CoG). IEEE, pp. 1-8.

Ontañón, S. (2013) 'The combinatorial multi-armed bandit problem and its application to real-time strategy games', Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 9(1), pp. 58-64.

Scheepers, D. and Pillay, N. (2025) 'A structure-based genetic programming generation constructive hyper-heuristic with transfer learning for combinatorial optimisation', Genetic Programming and Evolvable Machines, 27(1), p. 2.

Shleyfman, A., Komenda, A. and Domshlak, C. (2014) 'On combinatorial actions and CMABs with linear side information', in Frontiers in Artificial Intelligence and Applications, vol. 263: ECAI 2014. Amsterdam: IOS Press, pp. 825-[end page missing].

Stanescu, M., Barriga, N.A., Hess, A. and Buro, M. (2016) 'Evaluating real-time strategy game states using convolutional neural networks', 2016 IEEE Conference on Computational Intelligence and Games (CIG). Santorini, Greece: IEEE, pp. 1-7.

READ BUT NOT USED IN PROPOSAL
(in the library, not currently cited in the text)

Barriga, N.A., Stanescu, M. and Buro, M. (2018) 'Game tree search based on nondeterministic action scripts in real-time strategy games', IEEE Transactions on Games, 10(1), pp. 69-77.

microRTS AI Competition (no date) microRTS AI competition - microRTS. Available at: https://sites.google.com/site/micrortsaicompetition/microrts (Accessed: 29 May 2026).

Ontañón, S. (2016a) 'Experiments on learning unit-action models from replay data from RTS games', Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 12(2), pp. 9-14.

Ontañón, S. (2016b) 'Informed Monte Carlo tree search for real-time strategy games', 2016 IEEE Conference on Computational Intelligence and Games (CIG). Santorini, Greece: IEEE, pp. 1-8.

Ontañón, S. (2017) 'Combinatorial multi-armed bandits for real-time strategy games', Journal of Artificial Intelligence Research, 58, pp. 665-702.

Uriarte, A. and Ontañón, S. (2017) 'Single belief state generation for partially observable real-time strategy games', 2017 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, pp. 296-303.

Yang, Z. and Ontañón, S. (2018) 'Learning map-independent evaluation functions for real-time strategy games', 2018 IEEE Conference on Computational Intelligence and Games (CIG). IEEE, pp. 1-7.

Yang, Z. and Ontañón, S. (2021) 'Contextual combinatorial bandits in real-time strategy games', 2021 IEEE Conference on Games (CoG). IEEE, pp. 1-9.

[1] S. Ontanon, “The Combinatorial Multi-Armed Bandit Problem and Its Application to Real-Time Strategy Games,” *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.*, vol. 9, no. 1, pp. 58-64, Oct. 2013, doi: 10.1609/aiide.v9i1.12681.

[2] “microRTS AI Competition - Rules.” Accessed: May 30, 2026. [Online]. Available: https://sites.google.com/site/micrortsaicompetition/rules

[3] S. Huang, S. Ontañón, C. Bamford, and L. Grela, “Gym-μRTS: Toward Affordable Full Game Real-time Strategy Games Research with Deep Reinforcement Learning,” in *2021 IEEE Conference on Games (CoG)*, Copenhagen, Denmark: IEEE, 2021, pp. 671-678. doi: 10.1109/CoG52621.2021.9619076.
[4] Y. Jin, G. Slabaugh, and S. Lucas, “Adapter-RL: Adaptation of Any Agent Using Reinforcement Learning,” *IEEE Trans. Games*, vol. 18, no. 1, pp. 173-184, Mar. 2026, doi: 10.1109/TG.2025.3628606.
[5] S. Cui, S. Xu, A. He, Y. Wang, and B. Xu, “Empowering LLMs with Parameterized Skills for Adversarial Long-Horizon Planning,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, Jun. 2025, pp. 1-10. doi: 10.1109/IJCNN64981.2025.11227831.
[6] “microRTS AI Competition - microRTS.” Accessed: May 29, 2026. [Online]. Available: https://sites.google.com/site/micrortsaicompetition/microrts
[7] C. B. Browne *et al.*, “A Survey of Monte Carlo Tree Search Methods,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1-43, 2012, doi: 10.1109/TCIAIG.2012.2186810.
[8] Shleyfman Alexander, Komenda Antonín, and Domshlak Carmel, “On Combinatorial Actions and CMABs with Linear Side Information,” in *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2014. doi: 10.3233/978-1-61499-419-0-825.
[9] A. Komenda, A. Shleyfman, and C. Domshlak, “On Robustness of CMAB Algorithms: Experimental Approach,” in *Computer Games*, vol. 504, T. Cazenave, M. H. M. Winands, and Y. Björnsson, Eds., Cham: Springer International Publishing, 2014, pp. 16-28. doi: 10.1007/978-3-319-14923-3_2.
[10] D. Churchill, A. Saffidine, and M. Buro, “Fast Heuristic Search for RTS Game Combat Scenarios,” *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.*, vol. 8, no. 1, pp. 112-117, 2012, doi: 10.1609/aiide.v8i1.12527.
[11] M. Stanescu, N. A. Barriga, A. Hess, and M. Buro, “Evaluating real-time strategy game states using convolutional neural networks,” in *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, Santorini, Greece: IEEE, Sep. 2016, pp. 1-7. doi: 10.1109/CIG.2016.7860439.
[12] Y. Jin, X. Song, G. Slabaugh, and S. Lucas, “Partial Advantage Estimator for Proximal Policy Optimization,” *IEEE Trans. Games*, vol. 17, no. 1, pp. 158-166, Mar. 2025, doi: 10.1109/TG.2024.3408298.
[13] S. Goodfriend, “A Competition Winning Deep Reinforcement Learning Agent in microRTS,” in *2024 IEEE Conference on Games (CoG)*, Aug. 2024, pp. 1-8. doi: 10.1109/CoG60054.2024.10645610.
[14] MazzaAlessandro, *MazzaAlessandro/TMA*. (May 31, 2025). Java. Accessed: May 30, 2026. [Online]. Available: https://github.com/MazzaAlessandro/TMA
[15] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. Cambridge, MA: MIT Press, 1992.
[16] S. Liu, S. J. Louis, and C. A. Ballinger, “Evolving Effective Microbehaviors in Real-Time Strategy Games,” *IEEE Trans. Comput. Intell. AI Games*, vol. 8, no. 4, pp. 351-362, Dec. 2016, doi: 10.1109/TCIAIG.2016.2544844.
[17] X. Neufeld, S. Mostaghim, and D. Perez-Liebana, “Evolving Game State Evaluation Functions for a Hybrid Planning Approach,” in *2019 IEEE Conference on Games (CoG)*, IEEE, 2019, pp. 1-8.
[18] D. Scheepers and N. Pillay, “A structure-based genetic programming generation constructive hyper-heuristic with transfer learning for combinatorial optimisation,” *Genet. Program. Evolvable Mach.*, vol. 27, no. 1, p. 2, Dec. 2025, doi: 10.1007/s10710-025-09528-3.